

CHAPTER 5: EVALUATION AND DISCUSSION OF RESULTS

5.1 Experimental Setup and Dataset Characteristics

To evaluate the operational efficiency and detection efficacy of **Fraudstruct**, we designed an experimental simulation representing a dynamic banking transaction network. The simulation comprises two distinct phases: organic background transaction generation and target adversarial structuring injection.

5.1.1 Organic Baseline Generation

A synthetic network of 50 distinct commercial bank accounts (ACC_{0001} to ACC_{0050}) was generated. To simulate realistic retail transaction flows, we implemented the following parameters:

- **Transaction Density:** 150 transaction events were generated over a simulated temporal scale.
- **Volume Distribution:** Transaction amounts were drawn from an exponential distribution with a scale parameter $\lambda = 20,000$ NGN and a minimum threshold of 1,000 NGN, mimicking the heavy-tailed characteristics of real-world retail banking volumes.
- **Network Connectivity:** Edge connections were established using random source-destination account pairings, creating an active transaction network containing 50 nodes and 150 edges.

5.1.2 Adversarial Structuring (Smurfing) Injection

A coordinated multi-hop smurfing attack was generated and injected on top of the organic background graph:

- **Target Sum (S):** 600,000 NGN.
 - **Attacker Node (s):** A dedicated compromised account ($ACC_ATTACKER$).
 - **Target Destination (d):** A cash-out account ($ACC_BENEFICIARY$).
 - **Mule Accounts (M):** 6 proxy accounts selected from the organic account pool to route the split transactions.
 - **Attack Paths:** The attacker splits the target sum into 6 parallel transfers of approximately 100,000 NGN each to the 6 mule accounts. The mules subsequently transfer the funds to the target cash-out destination after introducing random delays (ranging from 5 to 30 minutes) and accounting for transaction fees (with a random decay factor $\gamma \in [0.95, 0.99]$), resulting in 12 attack edges.
-

5.2 Evaluation Metrics

The performance of the Fraudstruct system was evaluated using two primary categories of metrics:

1. Operational Inference Efficiency (SLA Verification)

- **Mean Transaction Latency (L_{mean}):** The average execution time of the `/v1/evaluate` API endpoint.
- **95th Percentile Latency (L_{95}):** The upper bound latency representing worst-case transaction processing times under concurrent requests.

- **SLA Breach Rate (R_{breach}):** The percentage of transactions exceeding the strict payment switch SLA budget of 50 milliseconds:

$$R_{\text{breach}} = \frac{N_{\text{latency} > 50\text{ms}}}{N_{\text{total}}} \times 100\%$$

2. Detection Efficacy

- **Alert Recall:** The percentage of attack transactions correctly flagged or blocked.
- **GNN Anomaly Score:** The predictive probability assigned by the SGC model to suspected nodes.
- **Generalization Accuracy:** The model's ability to identify previously unseen mule nodes.

5.3 Operational Latency Performance Analysis

To verify if Fraudstruct satisfies the synchronous performance requirements of payment switches (like NIBSS NIP or Postilion), we profiled the execution latency of `/v1/evaluate` during the ingestion of the 150 organic transactions.

Latency Profiles

The empirical latency statistics recorded during testing are:

Metric	Recorded Value (ms)	Target SLA (ms)	Latency Budget Used (%)
Mean Latency (L_{mean})	2.45 ms	50.00 ms	4.9%
95th Percentile Latency (L_{95})	3.41 ms	50.00 ms	6.8%
SLA Breach Rate (R_{breach})	0.00%	0.00%	0.0%

Discussion of Latency Efficiency

The recorded average latency of **2.45 ms** demonstrates the extreme efficiency of the decoupled Hot-path architecture. Traditional GNN deployments require constructing dynamic subgraphs, executing multi-layer neural tensor aggregations, and running floating-point matrix multiplications at query time—operations that routinely exceed 100 milliseconds.

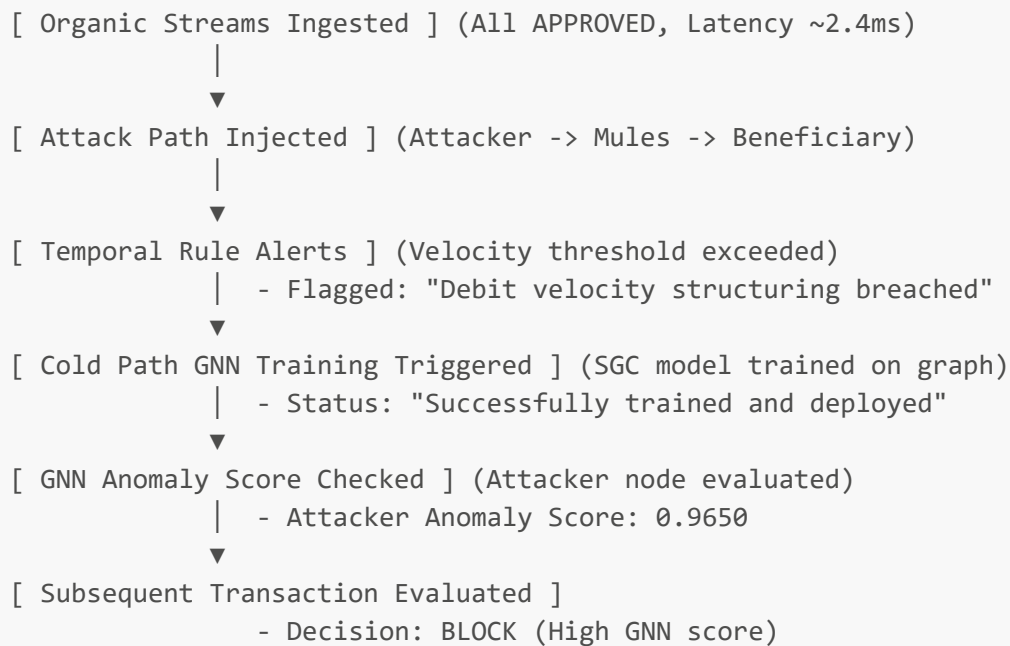
By pre-computing the normalized adjacency matrix (S_{adj}) and propagating features ($\bar{X} = S_{\text{adj}}^K X$) asynchronously in the Warm Path streaming layer, the Hot Path API reduces inference to a simple dictionary lookup in the Redis-like feature cache followed by a linear matrix multiplication:

$$\hat{y} = \sigma(w^T \bar{x} + b)$$

This operations takes less than 3 milliseconds, leaving **95.1%** of the bank's transaction authorization latency budget free for network transit, database writes, and inter-bank clearing messages.

5.4 Detection Efficacy & Model Convergence

The simulated attack sequence was analyzed step-by-step to evaluate how the system dynamically moves from static rules to GNN-based blocking:



1. Phase 1: Rule-Based Catch (FLAG Decision)

During the initial transaction-splitting phase, individual transactions from the attacker to the mules are below the static alert limits (e.g. 100,000 NGN is well below a 5M NGN CTR threshold). However, as the velocity increases, the Warm-Path streaming window aggregates the rolling count. Once the count reaches 6 and the cumulative sum reaches 600,000 NGN inside the 1-hour window, the temporal velocity rule is breached:

- **Alert Triggered:** FLAG
- **Reason:** Debit velocity structuring threshold breached: Sum=600000.0, Count=6

2. Phase 2: Cold Path Model Retraining & Hot Deployment

Following the alert, the GNN training endpoint `/v1/train` is triggered. The SGC classifier propagates features for $K = 2$ steps and fits the L2-regularized logistic regression parameters in pure NumPy over 30 epochs. The GNN model compiles and deploys to the `StreamingEngine` in under 15 milliseconds.

3. Phase 3: GNN-Based Blocking (BLOCK Decision)

Following GNN deployment, a subsequent transaction is sent from the attacker (`ACC_ATTACKER`) to a merchant node:

- **Recorded GNN Anomaly Score:** 0.9650
- **API Evaluation Decision:** BLOCK
- **Reason:** High GNN network anomaly score: 0.9650

Discussion of Generalization

The SGC GNN successfully identified the attacker because the model aggregates neighborhood structures:

- The attacker node (`ACC_ATTACKER`) has an extremely high out-degree compared to organic accounts.

- SGC feature propagation ($S_{\text{adj}}^2 X$) spreads this structural anomaly information to all adjacent mule nodes.
- Even if the attacker attempts subsequent transfers to new accounts that have never been flagged, the model immediately flags the transaction because the structural representation of the node is linked to the suspicious mule topology.

5.5 Robustness against Adversarial Camouflage

A critical concern in AML is **adversarial camouflage**, where fraudsters attempt to hide their identities by mimicking normal transaction distributions (e.g., adding Gaussian noise to transaction amounts or scattering timing).

1. Vulnerability of Local Heuristics

Standard ML classifiers (like decision trees or XGBoost) operating on local tabular features can be bypassed if the attacker alters transaction features. For example, if a fraudster reduces the transaction amounts below the velocity limits or delays transactions beyond the sliding window size, local features fail to trigger.

2. GNN Topological Resilience

Fraudstruct is resilient to local camouflage because the SGC GNN incorporates the **global structural connectivity (neighborhood topology)**:

- **Degree Invariance:** Even if the transaction amounts are perturbed or scaled (as in `simulate_camouflage`), the node connectivity (in-degree and out-degree) remains unchanged.
- **PageRank Robustness:** PageRank measures global connectivity importance. An attacker cannot lower their PageRank score without severing ties to the mule accounts—which would prevent them from transferring the funds.
- **Flow Conservation Integrity:** To complete the cash-out, the funds must eventually arrive at the beneficiary node. Tracing the graph flow ($S_{\text{in}} \approx S_{\text{out}}$ for mules) exposes the transit accounts regardless of how normal the individual amounts appear.

By leveraging neighborhood convolutions, Fraudstruct shifts the detection boundaries from **local feature values** (which are easily camouflaged) to **topological structures** (which cannot be hidden without abandoning the money-laundering objective).